Report - Team Deliverable 3 - The Pintos

Jovan Gelevski, John Burlando, Harry Carr, Mani Mir Mohammad Sadeghi, Ry Yebeyen, JP Gilliam

a. Requirements for Phase 1

- The system shall allow users to create an account using an email address and password.
- The system shall allow users to log in using the credentials specified during account creation.
- The system shall allow users to log out and terminate their active session.
- The system shall allow users to update account information such as email and password.
- The system shall allow users to permanently delete their account and associated data.
- The system shall allow users to add one or more vehicle profiles.
- The system shall allow users to manually enter vehicle details including make, model, year, mileage, fuel type, oil type, and recommended oil change interval.
- The system shall allow users to edit existing vehicle details.
- The system shall allow users to delete a vehicle profile.
- The system shall allow users to update the current mileage (odometer reading) of their vehicle at any time.
- The system shall allow users to log a maintenance event for a selected vehicle.
- The system shall allow users to select a service type (e.g., oil change, tire rotation, brake service, inspection).
- The system shall allow users to enter service date, odometer at time of service, and an optional description/notes section.
- The system shall allow users to edit an existing maintenance record.
- The system shall allow users to delete a maintenance record.
- The system shall display all maintenance records in chronological order.
- The system shall allow users to filter service history by service type.
- The system shall allow users to view detailed information for each maintenance entry.
- TuneUp provides automated reminder functionality to help users stay up to date with vehicle maintenance.
- The system shall allow users to configure maintenance reminders based on mileage intervals.
- The system shall allow users to configure maintenance reminders based on time intervals (e.g. every 6 months).
- The system shall display upcoming and overdue maintenance tasks on the dashboard.
- The system shall display all registered vehicles on the dashboard.
- The system shall display upcoming or overdue services.
- The system shall display recent maintenance entries.
- Pages shall load within 3 seconds under normal network conditions.
- Updates to local interface elements shall occur within 100 milliseconds.
    - Changed from previous requirement where timeframe was not concrete.
- Database updates shall complete within 3 seconds under normal network conditions.

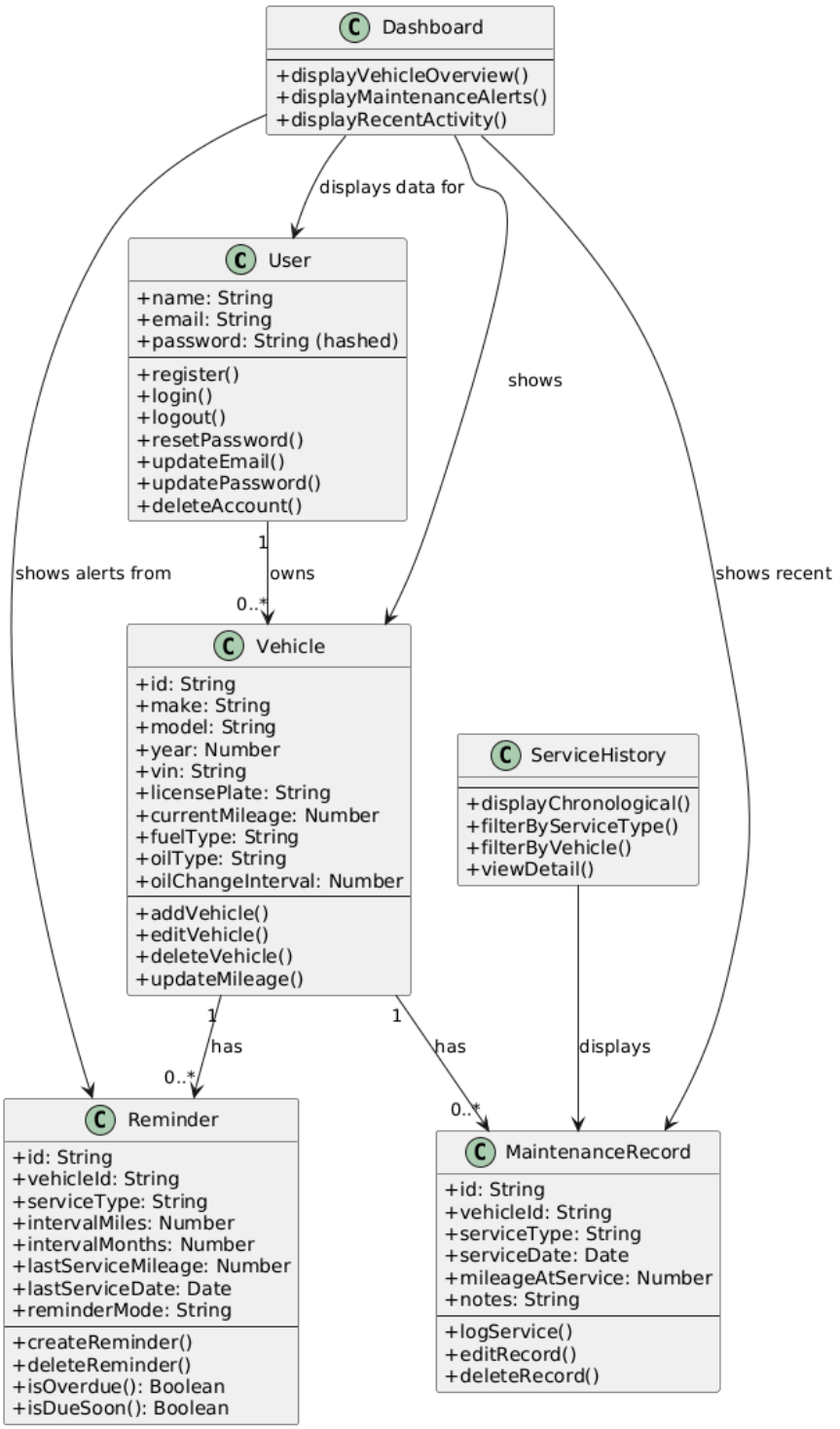
    - Changed from previous requirement where timeframe was again not specified concretely.
- Passwords shall be securely hashed with Bcrypt before being stored.
    - Changed from previous requirement where the hashing method was undefined.
- Users shall not be able to access or modify other users' data.
- All user input shall be validated to prevent common web vulnerabilities (e.g., SQL injection, cross-site scripting).
- The system shall function on desktop, tablet, and mobile devices.
- The interface shall allow new users to be able to create an account and log a maintenance record within 10 minutes without assistance.
- The system shall provide clear and descriptive error messages.
- The system shall maintain accurate and consistent storage of all user data.
- The system shall maintain at least 95% uptime during the project deployment period.
- The project shall use GitHub for version control.
- The system shall follow modular design principles to support future expansion.
- The system shall include setup and deployment documentation.
- The system shall handle user email and personal data in accordance with applicable privacy standards.
- The system shall allow users to request a password reset by providing their registered email address.
- The system shall generate a secure, time-limited password reset token upon a valid reset request.
- The system shall allow users to set a new password using a valid, unexpired reset token.
- The system shall invalidate a password reset token after it has been used or after it expires (1 hour).

Requirements Rationale:

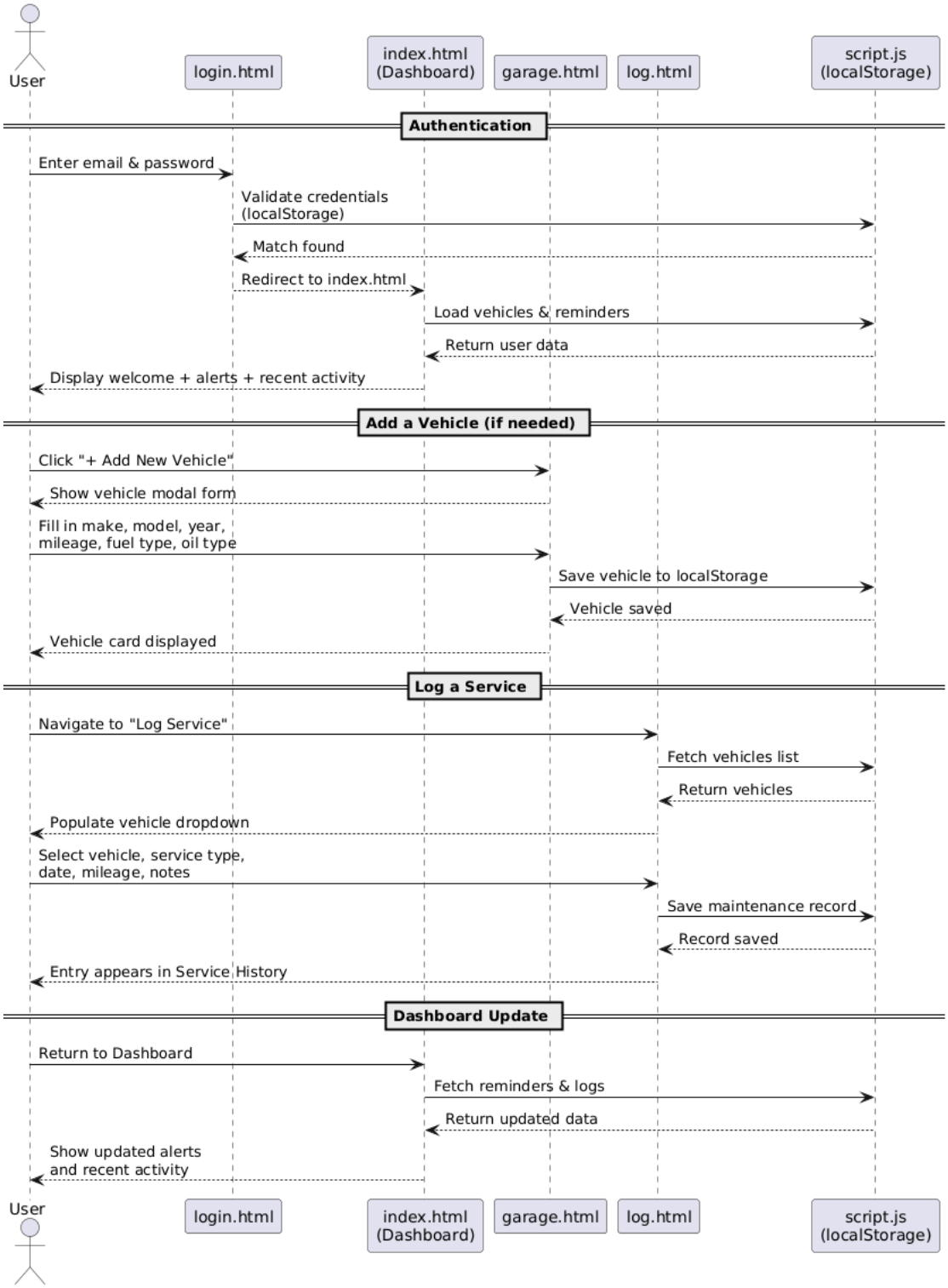
The requirements we set out to fulfill during phase one were mainly concerned with implementing core functionality such as the pages, the logic behind the pages, the data classes, data persistence, and other similar things. As phase one progressed and as it now comes to a close, we believe that we've ironed out a non-insignificant amount of requirements and that the project is getting close to the point where we can begin implementing the finer details. That being said, our scope from the start up until now has not changed significantly, and our requirements reflect this. A notable addition in this phase is the password reset functionality, which leverages the password_reset_tokens table already defined in our database schema and provides a complete forgot-password flow with secure token generation and expiry.

b. UML Diagrams

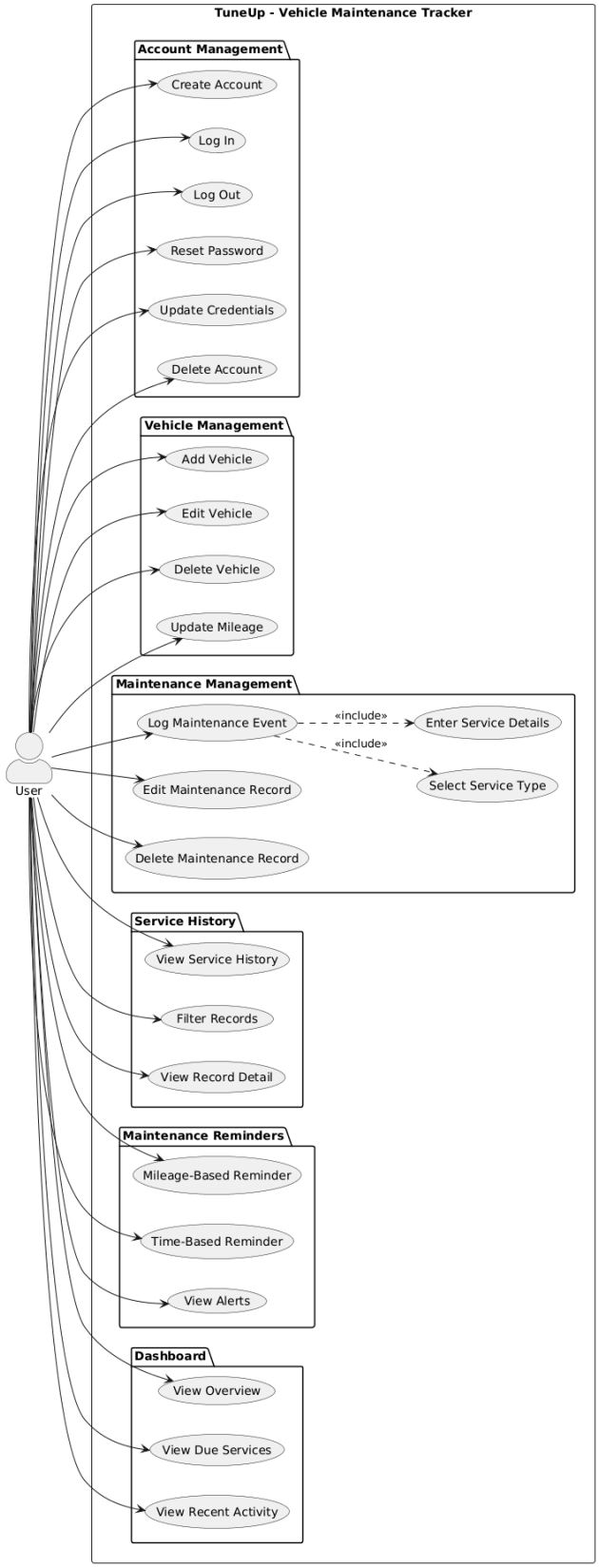
Class diagram -


Dashboard
+displayVehicleOverview()
+displayMaintenanceAlerts()
+displayRecentActivity()
displays data for
shows
shows alerts from
shows recent
User
+name: String
+email: String
+password: String (hashed)
+register()
+login()
+logout()
+resetPassword()
+updateEmail()
+updatePassword()
+deleteAccount()
1
owns
0..*
Vehicle
+id: String
+make: String
+model: String
+year: Number
+vin: String
+licensePlate: String
+currentMileage: Number
+fuelType: String
+oilType: String
+oilChangeInterval: Number
+addVehicle()
+editVehicle()
+deleteVehicle()
+updateMileage()
ServiceHistory
+displayChronological()
+filterByServiceType()
+filterByVehicle()
+viewDetail()
1
has
0..*
1
has
0..*
displays
Reminder
+id: String
+vehicleId: String
+serviceType: String
+intervalMiles: Number
+intervalMonths: Number
+lastServiceMileage: Number
+lastServiceDate: Date
+reminderMode: String
+createReminder()
+deleteReminder()
+isOverdue(): Boolean
+isDueSoon(): Boolean
MaintenanceRecord
+id: String
+vehicleId: String
+serviceType: String
+serviceDate: Date
+mileageAtService: Number
+notes: String
+logService()
+editRecord()
+deleteRecord()

Sequence diagram -



Use case diagram -



c. **User Manual**

This manual walks you through installing TuneUp on your computer and explains how to use every
feature in the app.

Installation and Setup

What you need installed first:

• A code editor (Visual Studio Code recommended)
• Node.js (includes npm, the Node package manager)
• MySQL Server 8.0 or newer
• Git

Step 1 - Download the project

Open a terminal and run:
git clone https://github.com/jovanlevsky/ThePintos.git
Then open the downloaded folder in Visual Studio Code.

Step 2 - Set up the database

Make sure MySQL is running on your computer. Open a terminal and log in to MySQL:
mysql -u root -p
Type your MySQL root password when asked. Then run this command to create the database:
SOURCE tuneupschema.sql;
This creates all the tables TuneUp needs. You can close this terminal when done.

Step 3 - Configure the backend

Navigate to the tuneup_backend folder inside the project. Create a new file called .env and paste
the following into it:

```
PORT=3000
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASS=<your-mysql-password>
DB_NAME=tuneup
JWT_SECRET=randomretgehdfhdsfhsdfhsd
JWT_EXPIRES_IN=7d
CORS_ORIGIN=http://127.0.0.1:5500/
```

Replace <your-mysql-password> with the root password you set for MySQL. Leave everything else as-is.

Step 4 - Install dependencies and start the server

Open a terminal in the tuneup_backend folder and run:
npm install
This downloads all the packages TuneUp needs. Once it finishes, start the server:
node server.jsYou should see TuneUp API running on port 3000 in the terminal. Keep this terminal open
while using the app.

Step 5 - Open the app in your browser

In Visual Studio Code, right-click on src/login.html and select "Open with Live Server" (you

may need to install the Live Server extension first). The app will open in your browser and you are
ready to go.

Using TuneUp

TuneUp has a navigation bar at the top of every page. Use it to move between the Dashboard, My
Garage, Log Service, Reminders, Profile, and About pages. The "Logout" button is also in the navigation bar.

Creating an Account

When you first open TuneUp, you will see the Login page.

1. Click "Sign up here!" below the login form.
2. Enter your name, email address, and a password (at least 8 characters).
3. Click "Create Account." You will be taken to the Dashboard.

Logging In

1. Enter your email and password on the Login page.
2. Click "Login." If your credentials are correct, you will be taken to the Dashboard.

Resetting a Forgotten Password

If you forget your password, you can reset it without logging in:

1. On the Login page, click "Forgot your password?"
2. Enter the email address you signed up with and click Send Reset Token.
3. A secure reset token will appear on screen. (In a real deployment this would be sent to your email.)
4. The token is automatically filled into the next form. Enter your new password, confirm it, and click Reset Password.
5. You will be taken back to the Login page. Sign in with your new password.

Tokens expire after 1 hour and can only be used once.

Dashboard

The Dashboard is the first page you see after logging in. It gives you a quick overview of everything important:

• **Maintenance Alerts** - Shows services that are overdue or coming up soon. Click "Log Now" to go straight to the Log Service page.
• **Recent Activity** - Lists your last 5 service entries so you can see what you have done recently.
• **My Garage** - Shows how many vehicles you have saved. Click "View Garage" to see all of them.

My Garage - Managing Your Vehicles

The My Garage page is where you add, view, edit, and delete your vehicles.

Adding a vehicle:

1. Click the "+ Add New Vehicle" button at the top right.
2. Fill in the required fields: Year, Make, Model, and Current Mileage. You can also add optional details like VIN, License Plate, Fuel Type, Oil Type, and Oil Change Interval.3. Click "Save Vehicle." The vehicle card will appear in your garage.

Editing or deleting a vehicle:

Each vehicle card has "Edit" and "Delete" buttons at the bottom. Click Edit to change any detail, or
Delete to remove the vehicle and all its records.
Each card also shows a status label: **Up to Date** (green), **Due Soon** (yellow), or **Overdue** (red), based on your reminders.

Log Service - Recording Maintenance

Use this page to record a service you have done on one of your vehicles.

1. Pick a vehicle from the dropdown.
2. Choose the date the service was done.
3. Select the type of service (Oil Change, Tire Rotation, Brake Pad Replacement, etc.).
4. Enter the mileage at the time of service.
5. Add any notes if you want (this is optional).
6. Click "Log Service." The entry will appear in your Service History on the right side of the page.

You can filter your service history by vehicle or by service type using the dropdowns above the list.
Each entry has Edit and Delete buttons if you need to make changes.

Reminders - Staying on Schedule

Reminders let TuneUp tell you when a service is due. You can set them based on mileage, time, or both.

1. Pick a vehicle. A reference box will appear showing the vehicle's current mileage and recent service logs.
2. Select the service type you want to be reminded about.
3. Choose how the reminder should work: by mileage only, by time only, or "whichever comes first."
4. Enter the interval (for example, every 5,000 miles or every 6 months).
5. Enter when this service was last done (mileage and date).
6. Click "Create Reminder."

Your reminders appear in the list on the right side of the page, and alerts show up on the Dashboard when a service is due soon or overdue.

Profile Settings

On the Profile page you can:

• **Update your name or email** - Change the fields and click Save Changes.
• **Change your password** - Enter your current password, then type and confirm a new one. A strength bar shows how strong your new password is (Weak, Fair, Good, or Strong).
• **Delete your account** - In the Danger Zone section, click Delete My Account. You will be asked to type your email to confirm. This permanently removes your account and all your data.**Logging Out**

Click "Logout" in the navigation bar at the top of any page. You will be returned to the Login page.

d. Test cases

In order to run the tests, navigate to the backend directory like before and open a terminal and run the command: “npx jest tuneup.test.js” It will install the jest test package and run the tests.

Here's an overview of the coverage:
Adding a vehicle -
We have a test suite where a reasonable set of cases is tested when it comes to adding a vehicle, which lie in two main categories: local data being sent to API, and API data being sent to local. The tests cover whether:
When a vehicle is sent to local from the API, the id is converted to a string, the values map properly (license_plate -> plate, mileage, fueltype), which values are defaulted to when the api sends null for oil interval, vin and plate.
When a vehicle is sent from local storage to API, the year is converted to a number format from string, the mileage is converted from a string to a number, that the previous values such as vin and oil interval properly map to null when empty, and that the other fields map properly.
When the site is relying on localStorage, the proper values are returned when certain things are queried, such as getVehicles returning an empty array when nothing is stored, whether saving and getting vehicles works, and whether reminders properly render.
When ids are being generated, they're being parsed correctly and only unique ids are being generated.
When tokens are generated and issued, they're properly handled when they're null or expired.
When maintenance logs are sent from API to local and vice versa, the fields properly map and fields properly default to expected values if they are null or empty.
When reminders are sent from API to local and vice versa, again, the fields properly map to each other (interval_miles->intervalMiles) and default value mapping is handled properly.
When getVehicleAlerts() is called, is it properly returning an empty array when a car that doesn't exist is called, is it returning an empty array when it has no reminders, is it showing proper flags when mileage exceeds a certain threshold (10,000) or time goes beyond a certain point (6 months)?
And lastly, when the initials function is called for the profile page, is it properly outputting a one or two character initial even when it has more than two names inputted or lots of excess whitespace, is it parsing lowercase letters to uppercase when converting them to an initial?

e. Reflection

So far, we've managed to build the repository to the point where the majority of main functionality has been implemented fully or close to fully. Development in a requirements-driven way has been a unique experience for us and so far, it's going really well. One of the things that went very well this phase was overall coordination between the team. We were able to accomplish what we set out to do for this first phase because of that. Another thing is handling development incrementally went well for us. Not trying to implement the whole system at once, but starting small and scalable and building off of that gave us a platform to work with. Some things that could have been improved in this phase were time management, spending more time on building out the backend, and doing more frequent check-ins with each other.

f. Member Contribution Table

Member Name	Contribution Description	Overall Contribution (%)	Note (if applicable
Jovan	Worked on merging	16.7%	

	functionality and building my garage and log service pages and some unit tests for the backend. Wrote half of the user manual and did the UML diagrams.		
JP	Worked on Dashboard logic and implementation, contributed to the code inspection documentation, helped develop test cases	16.7%	
John	Built framework site architecture, made contributions to the code inspection documentation.	16.7%	
Harry	Worked on the reminders tab	16.7%	
Ry	Worked on Login persistence and helped on inspection code	16.7%	
Mani	Implemented the password reset feature end-to-end	16.7%	